Paths and Pentagonal Gray Codes

Alexander Michos & Anjay Singla

December, 2024

Introduction

A *Gray Code* is a sequential ordering of the natural numbers $0, \ldots 2^n - 1$ such that their binary digits $\underbrace{\texttt{00...0}}_{n}, \texttt{00...1}, \ldots\ \texttt{11...1}$ (encoding base 2) differ at exactly *one* digit each increment. For example, a 2-bit Gray Code might take the form $0, 1, 3, 2$ since $0 = \texttt{00}$ adjusts one place to become $1 = \texttt{01}$, which differs by one digit from $3 = \texttt{11}$, which differs again by one digit from $2 = \texttt{10}$: we never had to roll two digits over to go from $\texttt{01}$ directly to $\texttt{10}$.

This simple adjustment to the ordering of a binary count turns out to be remarkably handy in computer science: oftentimes if you have some costly or complex function $f(x_1, \ldots x_n)$ which takes in n binary digits, it's easier to *update* the function by flipping exactly one of the x_i's at a time than it is to re-compute from scratch. Thus, if you're after the 2^n values $f(0, \ldots 0)$ up to $f(1, \ldots 1)$ you'd better use an ordering where you can update one digit at a time. Or, in electrical engineering, boolean circuits involving logical variables $x_1, \ldots x_n$ might be best analyzed / tested by negating one variable at a time: so in that context a Gray code enumeration might look like $x_1 \wedge x_2, x_1 \wedge \neg x_2, \neg x_1 \wedge \neg x_2, \neg x_1 \wedge x_2$.

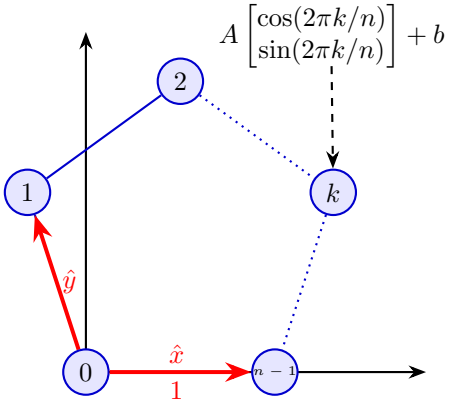
> *Exercise:* Notice that the 2-bit Gray Code $\texttt{00}, \texttt{01}, \texttt{11}, \texttt{10}$ forms a *loop* since the start and end differ only in one place. Continue this sequence to form a 3-bit Gray Code: append the digits $4, 5, 6, 7$ to the end in such a way that only one binary digit is flipped at each step. Does the result loop back to $\texttt{00}$? Form a conjecture on what the ending strings of an n-bit Gray Code look like.

It's a truly delightful fact that these sequences are actually in correspondence with *walks* along the vertices of n-dimensional cubes. Indeed, the 2^n binary strings involved are nothing more than an enumeration of the vectors in $\mathbb{R}^n$ having n coordinates filled with 0's and 1's: since two neighbors on the n-cube have a Euclidean distance of 1, the difference between any two digits in the binary strings they represent must be exactly 1 as well. For example, $\texttt{00}, \texttt{01}, \texttt{11}, \texttt{10}$ really ought to be thought of as a cycle from $(0, 0)$ to $(0, 1)$ to $(1, 1)$ to $(1, 0)$ which are the coordinates of a square in 2-dimensions, traversed clockwise from the origin. Trace the path of the 3-bit Gray code from the Exercise above to get a feeling for what this looks like geometrically in 3-space.

We were curious whether Gray codes could be formed out of pentagons instead of squares: would the golden ratio ϕ turn up in place of just 0's and 1's? And more generally, does the notion of walking along a closed cycle on a polyhedron give rise to other interesting cyclic, uniform-distance, Gray-like codes?

Regular Polygons

Given the relationship between the square and the 2-bit Gray Code, it's clear we ought to arrange n points in the shape of a regular n-gon and traverse them in clockwise order: starting with vertex 0 at $(0, 0)$ (ie $\texttt{00}$), and looping around to the last vertex at $(1, 0)$ (ie $\texttt{10}$). Along the way, we'll record how many steps (in terms of a fractional linear combination) we've taken in the two *basis* directions flanking the corner of the polygon, like so:



Where the coordinate of the kth vertex is given by a linear map A, plus some bias b applied to the "usual coordinates" $(\cos(2\pi k/n), \sin(2\pi k/n))$ of a regular n-gon:

$$A = -\frac{1}{2}\begin{bmatrix} 1 & \cot(\pi/n) \\ \cot(\pi/n) & -1 \end{bmatrix},\ b = \frac{1}{2}\begin{bmatrix} 1 \\ \cot(\pi/n) \end{bmatrix}$$

Exercise: Where in the plane is the point $(1, \cot(\pi/n))$? If A were factorized into the product of a reflection matrix and a *rotation matrix* what would the angle of rotation be?

Using these embeddings and some linear algebra, it's not hard to see that the kth step of our walk will require the following combination of $\hat{x}$ and $\hat{y}$ displacements (we'll use $[\square, \square]$ for $\hat{x}$,$\hat{y}$-coordinates):

$$\mathtt{C}[k] = \csc(\pi/n)\csc(2\pi/n)\sin(\pi k/n) \times \left[\sin\left(\frac{\pi\,(k-1)}{n}\right), \sin\left(\frac{\pi\,(k+1)}{n}\right)\right]$$

For example, by setting $n = 4$ clearly $\hat{x}$ and $\hat{y}$ become aligned with the usual coordinate axes, and we recover the displacements $[0,0], [0,1], [1,1], [1,0]$ from the above formula at $k = 0, 1, 2, 3$. Setting $n = 5$, we observe a smooth Gray-code walk using the golden ration $\phi = \frac{1+\sqrt{5}}{2}$ as a digit:

k	$\mathtt{C}_x[k]$	$\mathtt{C}_y[k]$
0	0	0
1	0	1
2	1	ϕ
3	ϕ	1
4	1	0

It's almost as if we're counting in a base-ϕ alphabet! (Or in the case of $n = 6$ where the displacements in $\mathtt{C}[k]$ come from $\{0, 1, 2\}$, in a sort of ternary alphabet.) The $n = 4$ case has a particularly nice algebraic simplification too;

$$\mathtt{C}[x] = \left[\frac{1}{2} - \frac{\sqrt{2}}{2}\cos\left(\frac{\pi}{2}x - \frac{\pi}{4}\right), \frac{1}{2} - \frac{\sqrt{2}}{2}\cos\left(\frac{\pi}{2}x + \frac{\pi}{4}\right)\right]$$

Which for no practical reason allows you to smoothly interpolate between the entries in the 2-bit Gray Code. It's an interesting question for the reader whether a "natural" *interpolation* formula exists for an n-bit Gray Code: it is well-known that these codes can be built recursively from the 2-bit code using simple concatenation and reflection operations. Perhaps it might be helpful to start with a 3-bit interpolation formula that was simply brute-forced by guessing a similar-*looking* cosine pattern:

$$\mathtt{C}[x] = \left[\frac{1}{2} - \frac{\cos(\pi/8)+\sin(\pi/8)}{2}\cos\left(\frac{\pi}{4}x - \frac{3\pi}{8}\right) + \frac{\cos(\pi/8)-\sin(\pi/8)}{2}\cos\left(\frac{3\pi}{4}x + \frac{7\pi}{8}\right),\right.$$
$$\left.\frac{1}{2} + \frac{\cos(\pi/8)-\sin(\pi/8)}{2}\cos\left(\frac{3\pi}{4}x + \frac{3\pi}{8}\right) - \frac{\cos(\pi/8)+\sin(\pi/8)}{2}\cos\left(\frac{\pi}{4}x + \frac{\pi}{8}\right),\ \ \frac{1}{2} - \frac{\sqrt{2}}{2}\cos\left(\frac{\pi}{2}x + \frac{\pi}{4}\right)\right]$$

Exercise: Prove that in a (binary) n-bit Gray code, the *average* value of each digit in the binary string is $1/2$. Why is there a $1/2$ added to the cos formula for $\mathtt{C}[x]$ above? Are the average values of the two columns in the table above equal? Without manipulating the trig formulas directly (besides the fact that $\cos(2\pi k/n)$ and $\sin(2\pi k/n)$ have average-value 0), show that this is true for every pentagonal Gray code. Why is this intuitive geometrically?

Dodecahedron

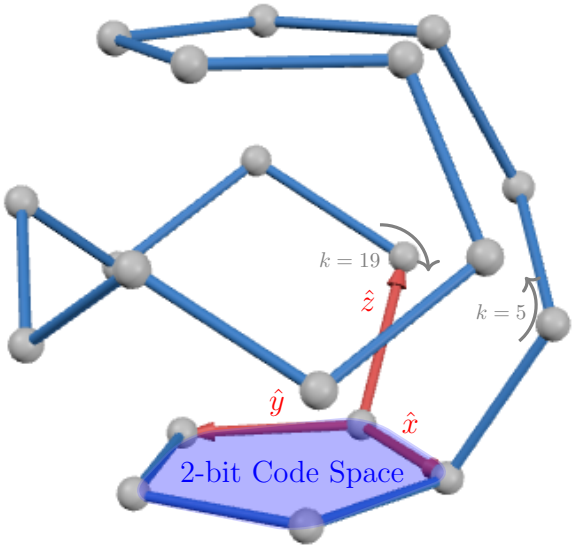
If a 3-bit Gray code is to a cube what a 2-bit Gray code is to a square, then a 3-bit pentagonal Gray code ought to emerge from walking along the edges of a Dodecahedron: a Platonic solid with pentagonal faces, 20 vertices and sporting a so-called *Hamiltonian Cycle*, which is a walk along the edges of the figure visiting each vertex without repetition.

But before we start walking around the Dodecahedron, we'll need to take the "usual coordinates" for the polyhedron and pass them through a series of shifts and rotations (like the A, b transformations above) to align them in a convenient fashion:

$$\mathcal{D} = \left\{(0, \pm\phi^{-1}, \pm\phi), (\pm\phi^{-1}, \pm\phi, 0), (\pm\phi, 0, \pm\phi^{-1}), (\pm 1, \pm 1, \pm)\right\} \rightsquigarrow \frac{1}{\sqrt{5}-1} R_x(\pi/10) \left(R_z(-\tan^{-1}(\phi^{-1})) \left(\mathcal{D} + \begin{bmatrix} 0 \\ 0 \\ \phi \end{bmatrix} \right) + \begin{bmatrix} 0 \\ \phi^{-1} \\ 0 \end{bmatrix} \right)$$

Which puts vertex 0 at the origin, vertex 4 at $(1, 0, 0)$ horizontally along the x axis, etc.

Ultimately, after carefully tracing out a Hamiltonian cycle (marked in blue segments and helper arrows in gray in the figure below), recording each coordinate, and transforming into $[\hat{x}, \hat{y}, \hat{z}]$ displacements, the following "dodecahedron code" was produced:



k	$\mathtt{C}_x[k]$	$\mathtt{C}_y[k]$	$\mathtt{C}_z[k]$
0	0	0	0
1	0	1	0
2	1	ϕ	0
3	ϕ	1	0
4	1	0	0
5	ϕ	0	1
6	1	0	ϕ
7	ϕ	1	ϕ^2
8	1	ϕ	ϕ^2
9	ϕ	ϕ^2	ϕ^2
10	ϕ^2	ϕ^2	ϕ^2
11	ϕ^2	ϕ	ϕ^2
12	ϕ^2	1	ϕ
13	ϕ^2	ϕ	1
14	ϕ^2	ϕ^2	ϕ
15	ϕ	ϕ^2	1
16	1	ϕ^2	ϕ
17	0	ϕ	1
18	0	1	ϕ
19	0	0	1

Geometrically, the walk has been structured so that it starts within the 2-bit code subspace (pentagonal code) we've already computed (highlighted in blue in the table and the figure) before softly meandering through the alphabet of symbols $0, 1, \phi, \phi^2$: never crossing sharply from 0 to ϕ or ϕ^2 in a beautiful Gray-code-like pattern.

> *Exercise:* Look for interesting patterns in the code! For example, there are *exactly* 5 0's, 5 1's, 5 ϕ's and 5 ϕ^2 in eac column, for a total of 20 symbols: what is the resulting per-column average? How do the last 3 rows and the first 3 rows compare?
>
> Using the double-angle formula for the tangent, show that $\tan^{-1}\left(\phi^{-1}\right) = \frac{1}{2}\tan^{-1}(2)$.

Mathematica & 4D

Manufacturing a 4-bit pentagonal Gray code would involve finding a Hamiltonian cycle along the *600* vertices of a 120-cell, or "hyperdodecahedron" (the 4-dimensional analog of the dodecahedron) not to mention orienting these points in space so that the $\hat{x}, \hat{y}, \hat{z}$ axes used in the 3-bit code are aligned in exactly the same way in 4-space as we expect, before determining the coded coordinates of each vertex in the walk! Fortunately, Clayton Shonkwiler at https://community.wolfram.com/groups/-/m/t/3274561, worked out exactly how to draw Hamiltonian cycles on a 120-cell, and so we'll be using his Mathematica code to generate the appropriate list of 600 `vertices`.

Much like how there are a pentagon of vertices lying on the bottom face of the dodecahedron, there are a set of 20 vertices forming a dodecahedron (`dodeclabels`) on the bottom "cell" of the hyperdodecahedron, interpreted along the 4th w axis. We'll want an affine map $x \mapsto Ax + b$ that carries the familiar $\hat{x}, \hat{y}, \hat{z}$ reference corner of that dodecahedron to the coordinates we worked out previously, and this can be done by just solving directly for A and b:

```
A = Table[a[i, j] , {i, 1, 4}, {j, 1, 4}]; B = Table[b[i], {i, 1, 4}];
expr = Join[
   Thread[A . vertices[[dodeclabels[[10]]]] + B == {0, 0, 0, 0}],
   Thread[A . vertices[[dodeclabels[[1x]]]] + B == {1, 0, 0, 0}],
   Thread[
    A . vertices[[dodeclabels[[1y]]]] + B == {-1/(1 + Sqrt[5]),
      Sqrt[1 + GoldenRatio^2]/2, 0, 0}],
   Thread[
    A . vertices[[dodeclabels[[1z]]]] + B == {-1/(
      1 + Sqrt[5]), -Sqrt[1 + GoldenRatio^2]/(2 Sqrt[5]),
      Sqrt[GoldenRatio/Sqrt[5]], 0}]
   ];
sol = Solve[expr, Join[Flatten[A], B]][[1]];
A = A /. sol; B = B /. sol;
expr = Map[
    Total[Power[A . vertices[[#[[1]]]] - A . vertices[[#[[2]]]], 2]] &,
     edges[[{4, 40, 100, 140}]]] // Expand;
sol2 = Solve[Thread[expr == 1], A[[All, 4]]][[1]];
A = A /. sol2; B = B /. sol2;
Print["Is the scale factor right? ", FullSimplify[Det[A]] == 1/(3 - Sqrt[5])^4];
```

After pushing all the coordinates through that map, we can identify $\hat{w}$, the new basis direction conferred by working in 4d, as the remaining vertex of the polyhedron at a distance of 1 from the origin. Then, as we walk along the list of vertices in our `hamiltoniancycle`, we change coordinates by multiplying by $[\hat{x}|\hat{y}|\hat{z}|\hat{w}]^{-1}$, and print out the resulting code!

```
mapped = Map[N[A . # + B] &, vertices];
{xbasisvec, ybasisvec, zbasisvec, wbasisvec} = {mapped[[dodeclabels[[1x]]]],
mapped[[dodeclabels[[1y]]]], mapped[[dodeclabels[[1z]]]], mapped[[331]]} ;
Print["Displacement Norms ", Norm[xbasisvec], Norm[ybasisvec], Norm[zbasisvec],
Norm[wbasisvec]]; code = Map[
   Inverse[Transpose[{xbasisvec, ybasisvec, zbasisvec, wbasisvec}]] .
     mapped[[#]] &, hamiltoniancycle];
```

As we can see on the right, the 4-bit Gray code is a dense and tedious brother to the 3-bit one: despite the alphabet of golden-ratio-based-symbols expanding drastically, the same "symbol distribution per column are equal" and "smooth transition from 0 to 1" properties are maintained. It's doubtful there's anything particularly interesting that can be gleaned from this messy, 600-word code, but hopefully the reader finds it a neat construction to think about nonetheless.

0	0	0	0
0	1	0	0
1	ϕ	0	0
ϕ	1	0	0
1	0	0	0
ϕ	0	1	0
1	0	ϕ	0
ϕ	1	ϕ^2	0
1	ϕ	ϕ^2	0
ϕ	ϕ^2	ϕ^2	0
ϕ^2	ϕ^2	ϕ^2	0
ϕ^2	ϕ	ϕ^2	0
ϕ^2	1	ϕ	0
ϕ^2	ϕ	1	0
ϕ^2	ϕ^2	ϕ	0
ϕ	ϕ^2	1	0
1	ϕ^2	ϕ	0
0	ϕ	1	0
0	1	ϕ	0
0	0	1	0
0	0	ϕ	1
0	0	1	ϕ
0	1	ϕ	ϕ^2
0	ϕ	1	ϕ^2
0	ϕ^2	ϕ	ϕ^2
0	ϕ^2	ϕ^2	ϕ^2
0	ϕ	ϕ^2	ϕ^2
1	ϕ^2	ϕ^3	ϕ^3
ϕ	ϕ^3	$2\phi^2$	$2\phi^2$
1	ϕ^3	ϕ^3	ϕ^3
ϕ	$2\phi^2$	$2\phi^2$	ϕ^3
ϕ^2	ϕ^4	$\sqrt{5}\phi^2$	$2\phi^2$
ϕ^3	$2\phi^3$	$3\phi^2$	ϕ^4
$2\phi^2$	$\sqrt{5}\phi^3$	$2\phi^3$	$2\phi^3$
ϕ^3	$2\phi^3$	ϕ^4	$3\phi^2$
$2\phi^2$	$\sqrt{5}\phi^3$	ϕ^4	$2\phi^3$
ϕ^3	$2\phi^3$	$2\phi^2$	ϕ^4
$2\phi^2$	$\sqrt{5}\phi^3$	$\sqrt{5}\phi^2$	ϕ^4
ϕ^4	ϕ^5	$3\phi^2$	$2\phi^3$
ϕ^4	ϕ^5	$2\phi^3$	$\sqrt{5}\phi^3$
ϕ^4	ϕ^5	$\sqrt{5}\phi^3$	$\sqrt{5}\phi^3$
$2\phi^3$	ϕ^5+1	ϕ^5	ϕ^5
$\sqrt{5}\phi^3$	$3\phi^3$	ϕ^5	ϕ^5+1
$2\phi^3$	ϕ^5+1	$\sqrt{5}\phi^3$	ϕ^5
$\sqrt{5}\phi^3$	$3\phi^3$	$\sqrt{5}\phi^3$	ϕ^5
$2\phi^3$	ϕ^5+1	$2\phi^3$	$\sqrt{5}\phi^3$
$\sqrt{5}\phi^3$	$3\phi^3$	$\sqrt{5}\phi^3$	$\sqrt{5}\phi^3$
$2\phi^3$	ϕ^5+1	$\sqrt{5}\phi^3$	$2\phi^3$
$\sqrt{5}\phi^3$	$3\phi^3$	ϕ^5	$\sqrt{5}\phi^3$
ϕ^5	$2\phi^4$	ϕ^5+1	ϕ^5
ϕ^5+1	$2\phi^4$	$3\phi^3$	ϕ^5
ϕ^5	$3\phi^3$	ϕ^5+1	$\sqrt{5}\phi^3$
$\sqrt{5}\phi^3$	ϕ^5+1	ϕ^5	$2\phi^3$
$2\phi^3$	ϕ^5	ϕ^5	$3\phi^2$
$3\phi^2$	ϕ^5	ϕ^5	$2\phi^3$
$2\phi^3$	ϕ^5+1	ϕ^5	$\sqrt{5}\phi^3$
ϕ^4	ϕ^5	$\sqrt{5}\phi^3$	$2\phi^3$
ϕ^4	ϕ^5	$2\phi^3$	$3\phi^2$
$2\phi^2$	$\sqrt{5}\phi^3$	ϕ^4	$\sqrt{5}\phi^2$
ϕ^3	$2\phi^3$	ϕ^4	$2\phi^2$
$2\phi^2$	$\sqrt{5}\phi^3$	$2\phi^3$	ϕ^4
$\sqrt{5}\phi^2$	$\sqrt{5}\phi^3$	$\sqrt{5}\phi^3$	ϕ^4
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\sqrt{5}\phi^2$	ϕ^4	ϕ^2	$2\phi^2$
$2\phi^2$	ϕ^4	ϕ^2	ϕ^3
ϕ^3	$2\phi^2$	ϕ	ϕ^2
ϕ^2	ϕ^3	1	ϕ
ϕ^2	ϕ^3	ϕ	1
ϕ^2	$2\phi^2$	ϕ^2	ϕ
ϕ	ϕ^3	ϕ^2	1
1	ϕ^3	ϕ^2	ϕ
0	ϕ^2	ϕ	1
0	ϕ^2	1	ϕ
0	ϕ	0	1
0	1	0	ϕ
0	0	0	1

Exercise: In a binary Gray code there is a clear notion of "unit distance" between `C`$[k]$ and `C`$[k+1]$: you can take the Euclidean dot-product $d(x, y) = (x-y)^\top(x-y)$ for example. Show that for any kind of polyhedral Gray code where a regular polyhedron is distorted via an affine map $x \mapsto Ax + b$, there exists a symmetric matrix Q such that $d_Q(x, y) = (x - y)^\top Q(x - y)$ gives "unit distance" between consecutive codewords.